Voting phase transitions are usually dealt with outside the balloting process, so in this case, you can drop the code related to the states `{Init, Regs, Vote, Done}` from the smart contract of chapter 3. We'll assume that an authority (such as an election commission) outside the chain manages the voting stages.

The version of the Ballot smart contract in listing 4.1 omits the states and includes only skeleton functions for brevity. Recall that you developed this solution in Solidity, using the design principles outlined in chapter 3. One more modifier validates that the voter is registered before they vote. Now is a good time to download all the files for the Ballot-Dapp and review them. You can find the complete listing of Ballot.sol in this codebase for this chapter. You'll use that listing as the smart contract for developing the Dapp.

> **NOTE** The `pragma` command shows a range of 0.4.22 to 0.6.0 to enable the Solidity features of these versions. For example, the features of 0.7.0 or 0.4.0 will not be enabled for the compilation of listing 4.1 and may throw an error if they are present.

Listing 4.1 Smart contract for the ballot use case (Ballot.sol)

```
pragma solidity >=0.4.22 <=0.6.0;
contract Ballot {

    struct Voter {

    }
    struct Proposal {

    }
    address chairperson;                                    ❶
    mapping(address => Voter) voters;                       ❶
    Proposal[] proposals;
```